

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ  
им. Н.Э. Баумана

Факультет “Информатика и системы управления”  
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по домашнему заданию  
“ Написание Телеграмм-бота на Python с использованием библиотеки telebot”

**Выполнил:**  
Студент группы ИУ5-34Б  
Клокова М.А.  
**Преподаватель:**  
Гапанюк Ю.Е.

Москва 2025

### **Постановка задачи**

1. Выберите язык программирования (который Вы ранее не изучали) и напишите по нему реферат с примерами кода или реализуйте на нем небольшой проект (с детальным текстовым описанием).
2. Реферат (проект) может быть посвящен отдельному аспекту (аспектам) языка или содержать решение какой-либо задачи на этом языке.
3. Необходимо установить на свой компьютер компилятор (интерпретатор, транспилятор) этого языка и произвольную среду разработки.
4. В случае написания реферата необходимо разработать и откомпилировать примеры кода (или модифицировать стандартные примеры).
5. В случае создания проекта необходимо детально комментировать код.
6. При написании реферата (создании проекта) необходимо изучить и корректно использовать особенности парадигмы языка и основных конструкций данного языка.
7. Приветствуется написание черновика статьи по результатам выполнения ДЗ. Черновик статьи может быть подготовлен группой студентов, которые исследовали один и тот же аспект в нескольких языках или решили одинаковую задачу на нескольких языках.

### **Описание программы**

Целью работы является приобретение навыков работы на Python с библиотекой telebot. В рамках задания реализован небольшой телеграмм-бот для регистрации и оформления заказов. Помимо библиотеки telebot используется библиотека sqlite для создания базы данных.

### **Код программы**

**main.py:**

```
import telebot  
  
import sqlite3  
  
import webbrowser  
  
from telebot import types
```

```
bot = telebot.TeleBot('8597939986:AAEgkXC04YpM2MlSuXQtnz7ZiBAU72yZbkE')
```

```
#создаем БД для регистрации пользователей с помощью sqlite3
```

```
def init_data_base():
```

```
    connection = sqlite3.connect('data_base.sql')
```

```
    cursor = connection.cursor()
```

```
    # пользователи
```

```
    cursor.execute('''
```

```
        CREATE TABLE IF NOT EXISTS users (
```

```
            id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
            telegram_id INTEGER UNIQUE NOT NULL,
```

```
            username TEXT,
```

```
            full_name TEXT,
```

```
            phone TEXT
```

```
        )
```

```
    ''')
```

```
    #заказы
```

```
    cursor.execute('''
```

```
        CREATE TABLE IF NOT EXISTS orders (
```

```
            id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
            user_id INTEGER NOT NULL,
```

```
            dish_name TEXT NOT NULL,
```

```
            quantity INTEGER DEFAULT 1,
```

```
            price REAL NOT NULL,
```

```
            status TEXT DEFAULT 'pending'
```

```
        )
```

```
    ''')
```

```
# статусы: pending/completed/cancelled

connection.commit()

connection.close()

init_data_base()

# меню продуктов

dish_list = {

    'pink_burger': {

        'name': '🍔 Розовый бургер',

        'description': 'Бургер с розовой булочкой',

        'price': 200,

        'category': 'main',

        'photo': 'burger.jpg'

    },

    'salad': {

        'name': '🥗 Фирменный салат',

        'description': 'Салат с крабовыми палочками',

        'price': 300,

        'category': 'salads',

        'photo': 'salad.jpg'

    },

    'classic_rolls': {

        'name': '🍱 Классические роллы',

        'description': 'Сет роллов с творожным сылом и крабовыми палочками',

        'price': 500,

        'category': 'main',
```

```

        'photo': 'rolls.jpg'
    },

    'dragon_fruit': {

        'name': '🍉 Дракони́й фрукт',

        'description': 'Спелый экзотический фрукт',

        'price': 100,

        'category': 'salads',

        'photo': 'fruit.jpg'
    },

    'choco_pie': {

        'name': '🥧 Чокопай',

        'description': 'Шоколадные пироженные',

        'price': 200,

        'category': 'dessert',

        'photo': 'choco_pie.jpg'
    },

    'cheese_cake': {

        'name': '🍰 Сырники',

        'description': 'Творожные сырники',

        'price': 300,

        'category': 'dessert',

        'photo': 'cheese_cake.jpg'
    }
}

# регистрация

def user_exists(telegram_id): # проверка

    connection = sqlite3.connect('data_base.sql')

    cursor = connection.cursor()

```

```

        cursor.execute('SELECT id FROM users WHERE telegram_id = ?', (telegram_id,))

        result = cursor.fetchone()

        connection.close()

        return result is not None

def register_user(telegram_id, username, full_name, phone):

    # НОВЫЙ ПОЛЬЗОВАТЕЛЬ

    try:

        connection = sqlite3.connect('data_base.sql')

        cursor = connection.cursor()

        cursor.execute('''

            INSERT INTO users (telegram_id, username, full_name, phone)

            VALUES (?, ?, ?, ?)

        ''', (telegram_id, username, full_name, phone))

        connection.commit()

        connection.close()

        return True

    except sqlite3.IntegrityError:

        return False # пользователь уже существует

    except Exception as e:

        print(f"Ошибка при регистрации: {e}")

        return False

def get_user_id(telegram_id):

    # id пользователя по telegram_id

    connection = sqlite3.connect('data_base.sql')

    cursor = connection.cursor()

    cursor.execute('SELECT id FROM users WHERE telegram_id = ?', (telegram_id,))

```

```

        result = cursor.fetchone()

        connection.close()

        return result[0] if result else None

# заказ добавляется

def add_order(user_id, dish_name, quantity=1):

    dish = dish_list.get(dish_name)

    if not dish:

        return False

    try:

        connection = sqlite3.connect('data_base.sql')

        cursor = connection.cursor()

        cursor.execute('''

            INSERT INTO orders (user_id, dish_name, quantity, price)

            VALUES (?, ?, ?, ?)

            ''', (user_id, dish['name'], quantity, dish['price'] * quantity))

        connection.commit()

        connection.close()

        return True

    except Exception as e:

        print(f"Ошибка при добавлении заказа: {e}")

        return False

# все заказы пользователя

def get_user_orders(user_id):

    connection = sqlite3.connect('data_base.sql')

    cursor = connection.cursor()

    cursor.execute('')

```

```

        SELECT dish_name, quantity, price, status

        FROM orders

        WHERE user_id = ?

        ORDER BY id DESC

    '', (user_id,))

orders = cursor.fetchall()

connection.close()

return orders


# сумма заказов пользователя

def get_order_total(user_id):

    connection = sqlite3.connect('data_base.sql')

    cursor = connection.cursor()

    cursor.execute('SELECT SUM(price) FROM orders WHERE user_id = ?', (user_id,))

    total = cursor.fetchone()[0]

    connection.close()

    return total or 0


# команды

@bot.message_handler(commands=['start']) # команда начала

def start(message):

    markup = types.ReplyKeyboardMarkup(resize_keyboard=True)

    if not user_exists(message.from_user.id):

        markup.add('📝 Зарегистрироваться')

    else:

        markup.add('🍷 Меню', '🛒 Мои заказы')

        markup.add('☎ Контакты', '📖 Помощь')

welcome_text = "Здравствуйте! 🤖"

```



```

if user_exists(message.from_user.id):

    welcome_text += "\nВы зарегистрированы. Можете сделать заказ."

bot.send_message(message.chat.id, welcome_text, reply_markup=markup)


# открытие стороннего сайта автора

@bot.message_handler(commands=['site', 'website'])

def site(message):

    webbrowser.open('https://github.com/Maria069/Labs_Sem3/tree/main')


# команда меню

@bot.message_handler(commands=['menu'])

def show_menu(message):

    markup = types.InlineKeyboardMarkup(row_width=2)

    # кнопки для продуктов

    buttons = []

    for dish_key, dish_info in dish_list.items():

        button_text = f"{dish_info['name']} - {dish_info['price']}₽"

        buttons.append(types.InlineKeyboardButton(button_text,
callback_data=f"order_{dish_key}"))

    # формируем ряды из кнопок с продуктами

    for i in range(0, len(buttons), 2):

        if i + 1 < len(buttons):

            markup.add(buttons[i], buttons[i + 1])

        else:

            markup.add(buttons[i])

```

```

# корзина

markup.add(types.InlineKeyboardButton("🛒 Перейти к корзине",
callback_data="view_cart"))

bot.send_message(

    message.chat.id,

    "🍷 *Меню:*\nВыберите продукт для заказа:",

    reply_markup=markup,

    parse_mode='Markdown'

)

#показывает историю заказов пользователя

@bot.message_handler(commands=['myorders'])

def my_orders(message):

    if not user_exists(message.from_user.id):

        bot.send_message(message.chat.id, "Вы еще не зарегистрированы.")

        return

    user_id = get_user_id(message.from_user.id)

    orders = get_user_orders(user_id)

    total = get_order_total(user_id)

    if not orders:

        bot.send_message(message.chat.id, "У вас еще нет заказов.")

        return

    orders_text = "📋 *Ваши заказы:*\n\n"

    for i, order in enumerate(orders, 1):

        dish_name, quantity, price, status = order

```

```

        status_emoji = '✅' if status == 'completed' else '⌚' if status == 'pending'
    else '❌'

    orders_text += f"{i}. *{dish_name}*\n"

    orders_text += f"    Количество: {quantity}\n"

    orders_text += f"    Сумма: {price}₽\n"

    orders_text += f"    Статус: {status_emoji} {status}\n\n"

orders_text += f"*Итого потрачено: {total}₽*"

bot.send_message(message.chat.id, orders_text, parse_mode='Markdown')

# текстовые сообщения
@bot.message_handler(content_types=['text'])
def handle_text(message):

    if message.text == '📝 Зарегистрироваться':

        if user_exists(message.from_user.id):

            bot.send_message(message.chat.id, "✅ Вы уже зарегистрированы!")

            return

# специальная отправка номера телефон

markup = types.ReplyKeyboardMarkup(resize_keyboard=True, one_time_keyboard=True)

markup.add(types.KeyboardButton("📞 Отправить номер телефона",
request_contact=True))

markup.add('❌ Отмена')

bot.send_message(

    message.chat.id,

    "Для регистрации нам понадобится ваш номер телефона.\n"

    "Нажмите кнопку, чтобы отправить его автоматически:",

```

```

        reply_markup=markup

    )

    bot.register_next_step_handler(message, process_phone_step)

elif message.text == '🍴 Меню':

    show_menu(message)

elif message.text == '🛒 Мои заказы':

    my_orders(message)

elif message.text == '📞 Контакты':

    bot.send_message(

        message.chat.id,

        "📞 *Наши контакты:*\n"

        "\n• Телефон: +7 (915) 852-25-53\n"

        "\n• Сайт: https://github.com/Maria069\n"

        "\n• Время работы: 10:00 - 23:00\n",

        parse_mode='Markdown'

    )

elif message.text == '❓ Помощь':

    bot.send_message(

        message.chat.id,

        "❓ *Помощь по боту:*\n\n"

        "\n• Зарегистрироваться – регистрация в системе\n"

        "\n• Меню – просмотр меню и заказ блюд\n"

        "\n• Мои заказы – история ваших заказов\n"

        "\n• Контакты – контактная информация\n\n"

        "\n*Команды:*\n"

```

```

        "/start - начать работу\n"

        "/menu - показать меню\n"

        "/myorders - мои заказы\n"

        "/site или /website- наш сайт",

        parse_mode='Markdown'

    )

elif message.text == '✕ Отмена':

    bot.send_message(message.chat.id, "Регистрация отменена.")

    start(message)

def process_phone_step(message):

    # получение номера телефона

    if message.content_type == 'contact':

        phone = message.contact.phone_number

        success = register_user(

            telegram_id=message.from_user.id,

            username=message.from_user.username,

            full_name=f"{message.from_user.first_name} {message.from_user.last_name or

''}",

            phone=phone

        )

    if success:

        bot.send_message(

            message.chat.id,

            f"✅ *Регистрация успешна!* \n\n"

            f"Добро пожаловать, {message.from_user.first_name}!\n"

            f"Теперь вы можете делать заказы.",

```

```

        parse_mode='Markdown',

        reply_markup=types.ReplyKeyboardRemove()

    )

    markup = types.ReplyKeyboardMarkup(resize_keyboard=True)

    markup.add('🍷 Меню', '🛒 Мои заказы')

    markup.add('📞 Контакты', '📖 Помощь')

    bot.send_message(message.chat.id, "Что закажете?", reply_markup=markup)

else:

    bot.send_message(message.chat.id, "❌ Ошибка при регистрации.")

else:

    bot.send_message(message.chat.id, "❌ Пожалуйста, отправьте номер телефона через кнопку.")

    start(message)

# inline кнопки

@bot.callback_query_handler(func=lambda call: True)

def handle_callback(call):

    if call.data.startswith('order_'):

        # обработка заказа блюда

        dish_key = call.data.replace('order_', '')

        dish = dish_list.get(dish_key)

        if not dish:

            bot.answer_callback_query(call.id, "Блюдо не найдено!")

            return

        user_id = get_user_id(call.from_user.id)

        if not user_id:

```

```

        bot.answer_callback_query(call.id, "Сначала зарегистрируйтесь!")

        return

# подтверждение заказа

markup = types.InlineKeyboardMarkup()

markup.add(

    types.InlineKeyboardButton("✅ Да, добавить",
callback_data=f"confirm_{dish_key}"),

    types.InlineKeyboardButton("❌ Отмена", callback_data="cancel_order")

)

"""

        bot.send_photo(call.message.chat.id, open(f"./{dish['photo']}",
'rb'), reply_markup=markup)

        bot.edit_message_text(

            chat_id=call.message.chat.id,

            message_id=call.message.message_id,

            text=f"*{dish['name']}*\n\n"

                f"{dish['description']}\n\n"

                f"Цена: *{dish['price']}₽*\n\n"

                f"Добавить в заказ?",

            parse_mode='Markdown'

        )

"""

        sent_message = bot.send_photo(

            call.message.chat.id,

            open(f"./photos/{dish['photo']}", 'rb'),

            caption=f"*{dish['name']}*\n\n{dish['description']}\n\nЦена:
*{dish['price']}₽*\n\nДобавить в заказ?",

            reply_markup=markup,

            parse_mode='Markdown'

```

```

    )

elif call.data.startswith('confirm_'):

    # подтверждение добавления в заказ

    dish_key = call.data.replace('confirm_', '')

    dish = dish_list.get(dish_key)

    user_id = get_user_id(call.from_user.id)

    bot.delete_message(call.message.chat.id, call.message.message_id)

    if add_order(user_id, dish_key):

        bot.answer_callback_query(call.id, f"✅ {dish['name']} добавлен в заказ!")

        # обновляем сообщение

        markup = types.InlineKeyboardMarkup()

        markup.add(

            types.InlineKeyboardButton("🔄 Продолжить заказ",
callback_data="continue_order"),

            types.InlineKeyboardButton("🛒 Оформить заказ", callback_data="checkout")

        )

    bot.send_message(

        call.message.chat.id,

        text=f"✅ *{dish['name']} добавлен в ваш заказ!*\n\n"

        f"Что дальше?",

        reply_markup=markup,

        parse_mode='Markdown'

    )

else:

```



```

        bot.answer_callback_query(call.id, "❌ Ошибка при добавлении заказа")

elif call.data == 'cancel_order':

    bot.delete_message(call.message.chat.id, call.message.message_id)

    bot.answer_callback_query(call.id, "Заказ отменен")

elif call.data == 'continue_order':

    # возвращаемся к меню

    show_menu(call.message)

elif call.data == 'checkout':

    # оформление заказа

    user_id = get_user_id(call.from_user.id)

    orders = get_user_orders(user_id)

    if not orders:

        bot.answer_callback_query(call.id, "Ваша корзина пуста!")

        return

    total = get_order_total(user_id)

    # формируем сообщение с заказом

    order_text = "🛒 *Ваш заказ:*\n\n"

    for order in orders:

        dish_name, quantity, price, _ = order

        order_text += f"• {dish_name} x{quantity} - {price}₽\n"

    order_text += f"\n*Итого: {total}₽*\n\n"

```

```
order_text += "Для оформления заказа свяжитесь с оператором по телефону +7 (XXX) XXX-XX-  
XX"
```

```
bot.edit_message_text(  
    chat_id=call.message.chat.id,  
    message_id=call.message.message_id,  
    text=order_text,  
    parse_mode='Markdown'  
)  
  
bot.answer_callback_query(call.id, "Заказ сформирован!")
```

```
elif call.data == 'view_cart':
```

```
    # просмотр корзины
```

```
    user_id = get_user_id(call.from_user.id)
```

```
    orders = get_user_orders(user_id)
```

```
    if not orders:
```

```
        bot.answer_callback_query(call.id, "Ваша корзина пуста!")
```

```
        return
```

```
    total = get_order_total(user_id)
```

```
    cart_text = "🛒 *Ваша корзина:* \n \n"
```

```
    for i, order in enumerate(orders, 1):
```

```
        dish_name, quantity, price, _ = order
```

```
        cart_text += f"{i}. {dish_name} x{quantity} - {price}₽ \n"
```

```
    cart_text += f" \n *Общая сумма: {total}₽"
```

```

markup = types.InlineKeyboardMarkup()

markup.add(

    types.InlineKeyboardButton("✅ Оформить заказ", callback_data="checkout"),

    types.InlineKeyboardButton("🔄 Продолжить покупки",
callback_data="continue_order")

)

bot.edit_message_text(

    chat_id=call.message.chat.id,

    message_id=call.message.message_id,

    text=cart_text,

    reply_markup=markup,

    parse_mode='Markdown'

)

# запуск бота

if __name__ == "__main__":

    bot.polling(none_stop=True)

}

```

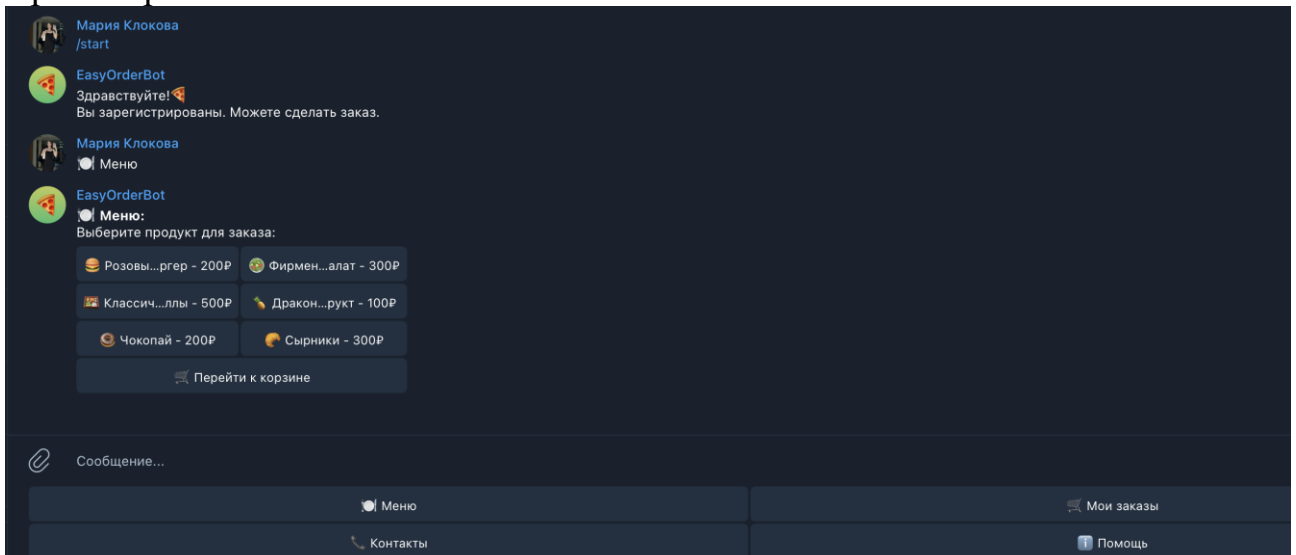
## Структура проекта

| DZ                                                                                                 |  |   |
|----------------------------------------------------------------------------------------------------|--|---|
| photos                                                                                             |  |   |
|  burger.jpg       |  | U |
|  cheese_cake.j... |  | U |
|  choco_pie.jpg    |  | U |
|  fruit.jpg        |  | U |
|  rolls.jpg        |  | U |
|  salad.jpg        |  | U |
| venv                                                                                               |  |   |
|  data_base.sql    |  | U |
|  main.py          |  | U |

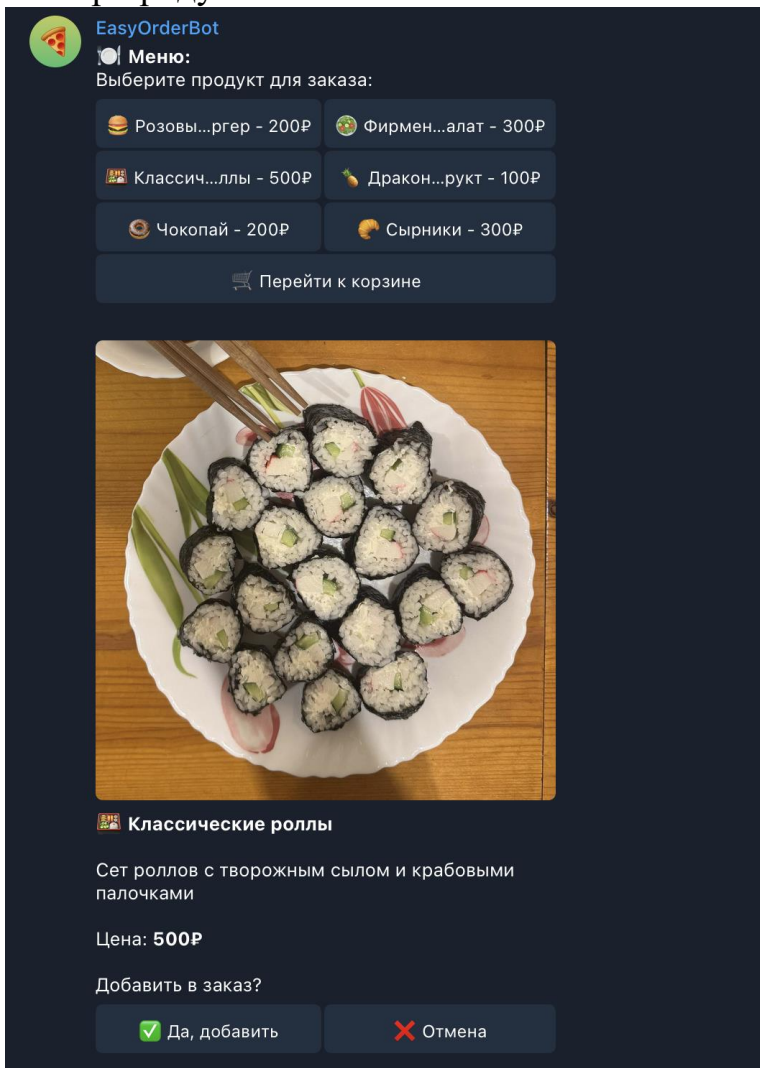
## Результат работы программы

- afeelu@mbp-stanislav DZ % source venv/bin/activate
- (venv) afeelu@mbp-stanislav DZ % python3 main.py

Просмотр меню:



Выбор продукта:



## Просмотр заказа:

**Мария Клокова**  
 Мои заказы

**EasyOrderBot**  
 **Ваши заказы:**

1.  **Классические роллы**  
Количество: 1  
Сумма: 500.0₽  
Статус:  pending

2.  **Драконий фрукт**  
Количество: 1  
Сумма: 100.0₽  
Статус:  pending

3.  **Драконий фрукт**  
Количество: 1  
Сумма: 100.0₽  
Статус:  pending

4.  **Сырники**  
Количество: 1  
Сумма: 300.0₽  
Статус:  pending

## Контакты:

**Мария Клокова**  
 Контакты

**EasyOrderBot**  
 **Наши контакты:**

- Телефон: +7 (915) 852-25-53
- Сайт: <https://github.com/Maria069>
- Время работы: 10:00 - 23:00

**GitHub**  
Maria069 - Overview  
Maria069 has 3 repositories available. Follow their code on GitHub.



## Помощь:



EasyOrderBot



Помощь по боту:

- Зарегистрироваться - регистрация в системе
- Меню - просмотр меню и заказ блюд
- Мои заказы - история ваших заказов
- Контакты - контактная информация

**Команды:**

[/start](#) - начать работу

[/menu](#) - показать меню

[/myorders](#) - мои заказы

[/site](#) или [/website](#) - наш сайт